ТОО ”Колледж Хекслет”

Лабораторная работа №8

Валидация входных данных и обработка ошибок

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Валидация входных данных и обработка ошибок.

Цель: Добавить в Internet Shop API проверку данных для операций создания и изменения товара, реализовать передачу ошибок через next(error) и создать централизованный middleware обработки ошибок.

Условие: RO4_2/lesson8/ЛР-8.pdf

Требования задания

• проверять данные POST /api/products;

• проверять переданные поля PATCH /api/products/:id;

• не создавать товар с некорректными данными;

• возвращать 400 для ошибки валидации;

• возвращать список обнаруженных ошибок;

• использовать AppError;

• передавать ошибки через next(error);

• возвращать 404 для отсутствующего товара;

• возвращать 404 для неизвестного маршрута;

• иметь централизованный errorHandler;

• возвращать 500 для неожиданной серверной ошибки;

• не раскрывать клиенту технические подробности внутренней ошибки.

Ход выполнения

POST проверяет все обязательные поля, PATCH — только переданные. Отклоняются пустое изменение, неизвестные поля и некорректные значения. Middleware возвращает список ошибок с 400. AppError передаётся через next(error), централизованный errorHandler формирует ответ; неизвестные маршруты дают 404, неожиданные ошибки — 500 без технических подробностей.

Проект: RO4_2/lesson8

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера: http://localhost:3000.

validators/productValidator.js

validators/productValidator.js

```
export function validateProductData(data, partial = false) {
    const errors = [];
    if (!data || typeof data !== 'object' || Array.isArray(data)) return ['JSON object required'];
    const fields = ['name', 'price', 'category', 'inStock'];
    if (partial && Object.keys(data).length === 0) errors.push('At least one field is required');
    for (const field of Object.keys(data)) {
        if (!fields.includes(field)) errors.push(`Unknown field: ${field}`);
    }
    if (!partial || 'name' in data) {
        if (typeof data.name !== 'string' || !data.name.trim() || data.name.length > 150) errors.push('name is required (max 150)');
    }
    if (!partial || 'price' in data) {
        if (!Number.isSafeInteger(data.price) || data.price <= 0 || data.price > 2147483647) errors.push('price must be a positive integer');
    }
    if (!partial || 'category' in data) { if (typeof data.category !== 'string' || !data.category.trim() || data.category.length > 100) errors.push('category is required (max 100)'); }
    if (!partial || 'inStock' in data) {
        if (typeof data.inStock !== 'boolean') errors.push('inStock must be boolean');
    }

    return errors;
}
```

Листинг 1. validators/productValidator.js

middleware/validateCreateProduct.js

middleware/validateCreateProduct.js

```
import { validateProductData } from '../validators/productValidator.js';
import { AppError } from '../errors/AppError.js';
export function validateCreateProduct(req, res, next) {
    const errors = validateProductData(req.body, false);
    if (errors.length) return next(new AppError('Validation failed', 400, errors));
    next();
}
```

Листинг 2. middleware/validateCreateProduct.js

middleware/validateUpdateProduct.js

middleware/validateUpdateProduct.js

```
import { validateProductData } from '../validators/productValidator.js';
import { AppError } from '../errors/AppError.js';
export function validateUpdateProduct(req, res, next) {
    const errors = validateProductData(req.body, true);
    if (errors.length) return next(new AppError('Validation failed', 400, errors));
    next();
}
```

Листинг 3. middleware/validateUpdateProduct.js

errors/AppError.js

errors/AppError.js

```
export class AppError extends Error {
    constructor(message, statusCode = 500, errors) {
        super(message);
        this.statusCode = statusCode;
        this.errors = errors;
    }
}
```

Листинг 4. errors/AppError.js

middleware/errorHandler.js

middleware/errorHandler.js

```
export function errorHandler(error, req, res, next) {
    if (res.headersSent) return next(error);
    let status = error.statusCode || (error.type === 'entity.parse.failed' ? 400 : 500);
    let message = error.type === 'entity.parse.failed' ? 'Invalid JSON' : error.message;
    if (['23505', 'P2002'].includes(error.code)) { status = 409; message = 'Record
already exists'; }
    if (['23503', 'P2003'].includes(error.code)) { status = 409; message = 'Related
record missing or still in use'; }
    if (error.code === 'P2025') { status = 404; message = 'Record not found'; }
    if (error.code === '23514') { status = 400; message = 'Constraint violation'; }
    if (status >= 500) console.error(error.message);
    res.status(status).json({ message: status >= 500 ? 'Internal server error' :
message,
        ...(error.errors ? { errors: error.errors } : {}) });
}
```

Листинг 5. middleware/errorHandler.js

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

• Полный CRUD-цикл

• Совместная фильтрация

• Уникальные id после удаления

HTTP-запросы

GET / → 200

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

GET /api → 200

GET /api/orders → 200

GET /api/products/3 → 200

GET /api/products/999 → 404

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?inStock=false → 200

POST /api/products → 403

POST /api/products → 403

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 400

PATCH /api/products/1 → 400

PATCH /api/products/1 → 400

GET /api/products/test/error → 500

POST /api/products → 201

GET /api/products/6 → 200

PATCH /api/products/6 → 200

PATCH /api/products/6 → 400

GET /api/products/6 → 200

PATCH /api/products/4/settings?notify=true → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

PATCH /api/products/6 → 404

DELETE /api/products/6 → 404

POST /api/products → 201

GET /api/unknown → 404

POST /api/orders → 201

GET /api/orders/1 → 200

PATCH /api/orders/1 → 200

GET /api/orders/999 → 404

Примеры ответов

POST /api/products → 201


```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true,
  "id": 6
}
```


PATCH /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```


DELETE /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.